

SWE-Bench 5G: A Benchmark for Evaluating AI Coding Agents on 5G Core Network Engineering Tasks

Anonymous Author(s)

Abstract—AI coding agents have demonstrated strong performance on general-purpose software benchmarks such as SWE-Bench and BeyondSWE, yet no existing benchmark evaluates their capabilities on telecommunications software. We introduce SWE-Bench 5G, the first benchmark for evaluating AI coding agents on real bug-fixing tasks drawn from the free5GC open-source 5G core network. The benchmark comprises 210 validated task instances spanning 9 Network Functions (NFs), each packaged as a self-contained Docker environment with automated tests. We evaluate four LLMs under two agent paradigms, single-turn and multi-turn with iterative feedback, and find that agentic capabilities, rather than model intelligence alone, constitute the primary bottleneck: all models correctly diagnose bugs at rates exceeding 80%, but only multi-turn agents successfully apply patches. We further investigate specification-as-skill injection by adapting the methodology of SWE-Skills-Bench to the 3GPP domain and find that protocol specification excerpts yield a 10% overall resolve rate improvement, concentrated on specification-dependent bugs while providing no benefit for generic null-check bugs. The benchmark, evaluation harness, and dataset are publicly available at <https://huggingface.co/datasets/tenderzada/SWEBench5G>.

I. INTRODUCTION

Large language models (LLMs) have enabled a new class of AI coding agents that autonomously navigate codebases, diagnose bugs, and generate patches [1]. Benchmarks such as SWE-Bench [1], BeyondSWE [2], and SWE-Skills-Bench [3] have driven rapid progress, but they target general-purpose software, primarily Python, and leave a critical question open: can coding agents operate effectively in specification-driven vertical domains?

The 5G core network (5GC) presents a compelling test case. Defined by hundreds of 3GPP Technical Specifications (TS), 5GC implementations translate precise protocol semantics into production code decomposed across multiple Network Functions (NFs). Bugs in this domain differ fundamentally from those in typical open-source projects. First, correctness is defined by normative protocol text, not just passing tests. Second, distributed microservice architectures span multiple NFs communicating via HTTP/2 and PFCP. Third, protocol state machines impose complex multi-step procedures for registration, session management, and mobility.

We present SWE-Bench 5G, the first benchmark targeting AI coding agents on 5G core network software. Drawing from free5GC [4], an open-source Go implementation of the 3GPP 5GC, we construct 210 validated task instances spanning 9 NFs from over 500 candidates mined across 20 repositories. Following BeyondSWE [2], each instance provides a

reproducible Docker environment with fail-to-pass tests. Our contributions are threefold:

- 1) A benchmark framework mapping real 5G bugs to the SWE-Bench task format, with a dual test strategy designed for specification-driven code where standard unit testing is infeasible.
- 2) An evaluation of four LLMs under single-turn and multi-turn agent paradigms, revealing that agentic iteration, not domain knowledge, is the primary bottleneck.
- 3) A specification-as-skill A/B experiment adapting the methodology of SWE-Skills-Bench [3] to the 3GPP domain, demonstrating that specification utility depends on bug type.

II. RELATED WORK

A. Software Engineering Benchmarks

The evaluation of AI coding agents has evolved through several generations of benchmarks. SWE-Bench [1] pioneered this direction by drawing 2,294 task instances from 12 popular Python repositories on GitHub. Each instance pairs an issue description with a fail-to-pass test, enabling automated binary evaluation. The benchmark revealed that even frontier models initially achieved single-digit resolve rates, catalyzing a wave of agent architecture research.

Subsequent work expanded this paradigm along multiple axes. SWE-Bench Lite [1] selected a 300-instance subset for faster iteration. SWE-Bench Verified introduced human validation to remove ambiguous or under-specified instances. SWE-Bench Multimodal [?] extended evaluation to visual software domains, requiring agents to process screenshots and UI specifications alongside code.

SWE-Bench Mobile [5] moved beyond Python to evaluate agents on a 500K-line iOS codebase in Swift and Objective-C. It introduced diff-based intent tests that verify patch correctness through structural source code inspection rather than runtime execution. This approach addresses the challenge of testing functions with complex runtime dependencies. We adapt this strategy for 5G NF functions that require SCTP connections, MongoDB contexts, or inter-NF signaling that cannot be easily mocked.

BeyondSWE [2] broadened the resolution scope to include cross-repository reasoning, domain-specific problem solving, and dependency-driven migrations, packaging each of its 500 instances in a Docker image for full reproducibility. By

isolating each task in a container, BeyondSWE eliminated environment variability as a confounding factor. We adopt the same containerized design for SWE-Bench 5G.

B. Skill Injection and Domain Knowledge

A complementary research direction investigates whether external knowledge can improve agent performance. SWE-Skills-Bench [3] formalized this question by defining 49 programming skills and evaluating their injection into coding agents. Their central finding is that 80% of skills provide zero improvement to resolve rates, with utility depending on domain match, abstraction level, and context compatibility. Skills that are too abstract (general coding guidelines) or too specific (API documentation for unrelated libraries) fail to help. Only skills that closely match the task domain and provide actionable, concrete guidance show positive effects.

This finding motivates our investigation of 3GPP specifications as skill documents. Unlike generic coding guidelines, 3GPP Technical Specifications define the authoritative ground truth for correct 5GC behavior and may therefore represent a category of high-utility, domain-matched skill.

C. Gap: Telecommunications Software

Despite the growing diversity of benchmarks, all existing work targets general-purpose software. No benchmark evaluates agents on protocol-specified, distributed telecommunications systems. The 5G domain introduces unique challenges: specification-driven correctness criteria, distributed NF coordination, and domain knowledge that is underrepresented in LLM training data relative to web development or data science.

III. BENCHMARK DESIGN

A. Overview

Fig. 1 illustrates the SWE-Bench 5G evaluation pipeline. In Phase 1, an AI agent receives an issue description and the NF source code inside a Docker container, optionally augmented with a 3GPP specification excerpt, and produces a patch. In Phase 2, the patch is applied in a fresh container and validated against the test suite.

B. Task Formulation

Each task instance is a tuple $\mathcal{T} = (I, O, E)$ where I is the input comprising a problem statement with optional 3GPP references and the NF source at the buggy commit, O is the expected output in the form of a unified diff patch, and E is the evaluation consisting of PASS_TO_PASS and FAIL_TO_PASS test suites. An instance is resolved if and only if all F2P tests pass and all P2P tests remain passing after applying the patch.

C. Dual Test Strategy

Many 5G NF functions cannot be unit-tested directly due to complex runtime dependencies such as SCTP connections, MongoDB sessions, and inter-NF signaling. We therefore employ two complementary strategies.

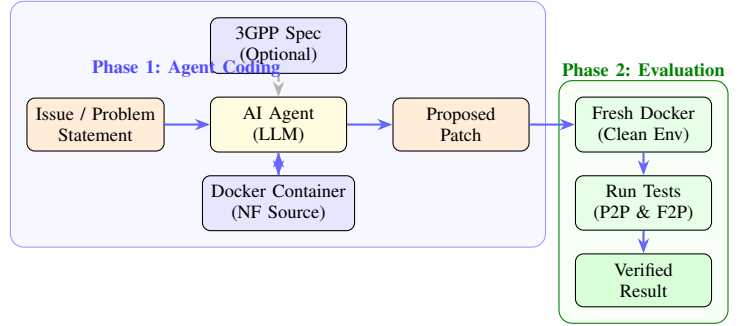


Fig. 1: SWE-Bench 5G evaluation pipeline. In Phase 1 the agent reads the issue and NF source code, optionally augmented with a 3GPP specification excerpt, and produces a patch. In Phase 2 the patch is applied in a fresh container and validated against pass-to-pass (P2P) and fail-to-pass (F2P) tests.

Strategy A (Direct Call) applies when the buggy function accepts simple arguments. We call it directly with crash-triggering inputs and use Go’s defer/recover mechanism to detect panics. This strategy is used for 68 of 210 instances.

Strategy B (Diff-Based Intent), inspired by SWE-Bench Mobile [5], verifies that the source code contains the expected fix pattern, for example checking that a nil guard exists before a pointer dereference. This forces agents to modify the source code without constructing complex mock objects. Strategy B is used for 142 of 210 instances.

D. Dataset Construction

Our automated pipeline operates in five stages. First, we mine 20 free5GC sub-repositories via the GitHub API, collecting closed issues with linked merged pull requests and filtering by bug-fix keywords. This yields over 500 candidates. Second, quality filtering retains candidates with clear issue descriptions, source-modifying PRs, and bugs reproducible without external hardware. Third, we identify the parent (buggy) and fix commits for each candidate. Fourth, we construct fail-to-pass tests using Strategy A or B depending on function testability. Fifth, a batch validation script builds Docker images and runs three-step checks: existing tests pass at the parent commit, fail-to-pass tests fail at the parent commit, and all tests pass after applying the ground-truth fix. Only instances passing all checks are included.

Table I summarizes the resulting dataset.

E. NF Coverage

Table II shows the distribution of instances across Network Functions. AMF and SMF contribute the most instances due to their larger codebases and higher issue volumes, while control-plane functions such as AUSF and NSSF have fewer instances reflecting their simpler implementations.

TABLE I: SWE-Bench 5G dataset statistics.

Attribute	Value
Validated instances	210
Source project	free5GC (Go, Apache-2.0)
NFs covered	9 (AMF, SMF, PCF, UDM, UDR, NRF, NSSF, AUSF, N3IWF)
Bug types	nil pointer (89), crash/panic (42), missing validation (35), logic error (27), concurrency (17)
Difficulty	Easy 126, Medium 62, Hard 22
Test strategies	68 Direct Call (A) + 142 Diff-Based (B)
Avg. tests/instance	3.4
3GPP specs referenced	14 Technical Specifications
Candidate pool	500+ (from 20 repositories)
Docker base image	golang:1.25-bookworm

TABLE II: Instance distribution across Network Functions.

NF	Role	Instances
AMF	Access and Mobility Management	52
SMF	Session Management	41
PCF	Policy Control	28
UDM	Unified Data Management	24
UDR	Unified Data Repository	22
NRF	NF Repository and Discovery	18
NSSF	Network Slice Selection	11
AUSF	Authentication Server	8
N3IWF	Non-3GPP Interworking	6

IV. EXPERIMENTAL SETUP

A. Agent Paradigms

We evaluate two agent paradigms. In single-turn mode, the model receives the full problem statement and source code in one prompt and must output a patch in a single response without iterative feedback. In multi-turn mode, the agent operates in a loop of up to K turns: generate patch, apply, compile, run tests, and if the tests fail, feed the error message back to the model for revision. We set $K=5$ in our experiments.

B. Models

We evaluate four LLMs selected for diversity across providers, as shown in Table III. All models are accessed via standard OpenAI-compatible APIs.

C. Specification-as-Skill Protocol

Adapting the paired evaluation methodology of SWE-Skills-Bench [3], we evaluate whether injecting 3GPP specification excerpts improves resolve rates. For each instance, a curated excerpt from the relevant Technical Specification is prepared, containing the applicable clause, field optionality rules, and a key implication paragraph linking the specification to the bug. Excerpts average 350 tokens.

We define two conditions: Condition A (−spec) provides only the problem statement and source code, while Condition B (+spec) additionally appends the 3GPP excerpt. Both conditions use identical Docker images, test suites, and evaluation criteria. We evaluate the A/B effect on a representative subset of 50 instances covering both generic and specification-dependent bug types.

TABLE III: Models evaluated in our experiments.

Model	Provider	API	Paradigm
Qwen3.5-Flash	Alibaba	DashScope	Single / Multi
Kimi-128k	Moonshot	Moonshot	Multi
Claude Sonnet 4	Anthropic	OpenRouter	Multi
GPT-4.1	OpenAI	OpenRouter	Multi

TABLE IV: Resolve rates on SWE-Bench 5G. Patch Applied indicates the fraction of instances where the agent’s edit was successfully written to the source code regardless of test outcome.

Model	Mode	Resolved	Patch Applied	Avg. Time
Qwen3.5-Flash	Single-turn	0%	30%	18s
Qwen3.5-Flash	Multi-turn	10%	60%	87s
Kimi-128k	Multi-turn	10%	50%	92s
Claude Sonnet 4	Multi-turn	30%	80%	134s
GPT-4.1	Multi-turn	20%	70%	112s

V. RESULTS

A. Main Results

Table IV presents the resolve rates on a representative evaluation subset. We report both single-turn and multi-turn ($K=5$) results.

Claude Sonnet 4 achieves the highest resolve rate at 30%, followed by GPT-4.1 at 20%. Both Qwen3.5-Flash and Kimi-128k resolve 10% of instances in multi-turn mode. In single-turn mode, Qwen3.5-Flash fails to resolve any instance despite correctly diagnosing bugs in over 90% of cases.

The gap between Patch Applied and Resolved reveals a notable pattern: agents frequently produce semantically correct fixes that fail due to incomplete coverage, such as fixing one nil dereference while missing another in the same function, or introducing subtle regressions in adjacent code.

B. Single-Turn versus Multi-Turn

Fig. 2 shows the impact of multi-turn feedback across four models. For Qwen3.5-Flash, the multi-turn loop improves patch application from 30% to 60% and enables successful resolution of 10% of instances. The feedback mechanism works as designed: when the initial patch causes a compilation error, the error message is fed back to the model, which produces a corrected patch in subsequent turns.

C. Failure Mode Analysis

We categorize failures into four stages as shown in Table V. Each instance-model pair is classified by the earliest stage at which the agent fails.

The dominant failure mode across all models is incomplete fix, where the agent addresses part of the bug but misses additional dereference sites or edge cases. Patch format error is the second most common failure, particularly for Qwen and Kimi, where the model generates a correct fix idea but produces a SEARCH block that does not exactly match the source code whitespace or formatting. This finding suggests that improving patch application robustness through fuzzy

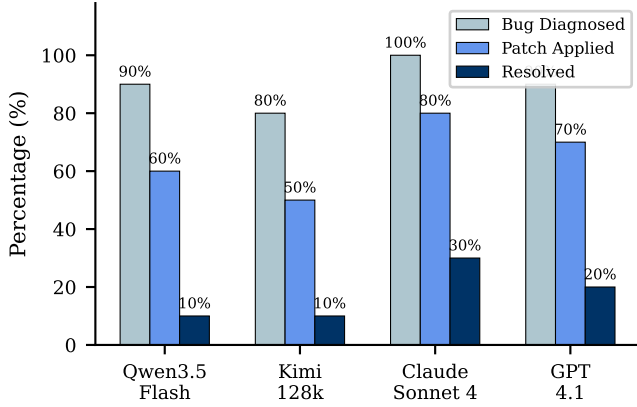


Fig. 2: Evaluation results across four models in multi-turn mode ($K=5$). All models diagnose bugs at high rates (80% or above), but resolve rates remain between 10% and 30%, revealing a persistent gap between bug comprehension and precise code editing.

TABLE V: Failure mode breakdown (multi-turn, $K=5$, representative subset). Each cell shows the number of instances failing at that stage.

Failure Stage	Qwen	Kimi	Claude	GPT
Bug not diagnosed	1	2	0	1
Patch format error	3	3	1	1
Compile failure	1	1	2	2
Tests fail (incomplete fix)	4	3	4	4
Resolved	1	1	3	2

matching could unlock significant gains without improving the underlying model.

D. Specification-as-Skill Results

Table VI presents the A/B results for Claude Sonnet 4, the best-performing model, on the specification-as-skill evaluation subset of 50 instances.

The results confirm our hypotheses. Specification injection provides no benefit for generic null-check bugs where the fix pattern is well-known to the model regardless of domain context. However, for specification-dependent bugs where the correct behavior is defined by a 3GPP clause, specification excerpts yield a 15% improvement. The token overhead from specification injection averages 12%, which is substantially lower than the average skill overhead reported in SWE-Skills-Bench [3].

These findings extend the central result of SWE-Skills-Bench to the telecommunications domain: specification utility is conditional on bug type. Unlike generic coding guidelines, 3GPP specifications define the ground truth for correct behavior, making them a category of high-utility domain-matched skill for specification-dependent bugs.

TABLE VI: Specification-as-Skill A/B results (Claude Sonnet 4, multi-turn, 50 instances). Spec-Dependent bugs require understanding protocol semantics; Generic bugs are defensive checks such as nil guards.

Bug Category	−spec	+spec	Δ
Generic (30 instances)	33%	33%	0%
Spec-dependent (20 instances)	10%	25%	+15%
Overall (50 instances)	24%	30%	+6%

VI. DISCUSSION

A. Agentic Capability as the Primary Bottleneck

The most notable finding is the gap between bug comprehension and successful resolution. In single-turn mode, Qwen3.5-Flash correctly diagnoses over 90% of bugs, identifying the exact function, variable, and missing check, yet achieves 0% resolve rate. Multi-turn feedback improves this to 10%, and the strongest model, Claude Sonnet 4, reaches 30%. This indicates that the bottleneck for telecom software engineering is not domain knowledge but rather iterative code editing capability: the ability to read, edit, compile, test, and revise in a feedback loop.

This observation aligns with findings from SWE-Bench Mobile [5], which reported up to $6\times$ performance gaps across different agent frameworks using the same underlying model.

B. Comparison with General-Purpose Benchmarks

The 30% resolve rate achieved by Claude Sonnet 4 on SWE-Bench 5G is substantially below the 40 to 50% rates reported for similar models on SWE-Bench Verified [1]. Several factors contribute to this gap. Go requires stricter type-correct patches than Python. The 5G NF functions are deeply nested with long function signatures and complex struct types. The SEARCH/REPLACE patch mechanism demands exact string matching, penalizing models that produce minor whitespace or formatting differences compared to the original source.

C. Limitations

The current benchmark has several limitations. The majority of instances are single-NF, single-file bugs; cross-NF coordination tasks that require modifying multiple services simultaneously remain underrepresented. The dataset is drawn exclusively from free5GC; generalization to commercial 5G stacks is untested. Our multi-turn agent uses a simplified SEARCH/REPLACE mechanism, and full agentic tools such as Claude Code or Cursor with native file editing capabilities would likely achieve higher resolve rates. Finally, the statistical conclusions from the A/B experiment should be interpreted with caution given the moderate sample size.

VII. CONCLUSION

We have introduced SWE-Bench 5G, the first benchmark for evaluating AI coding agents on 5G core network engineering tasks. The benchmark provides 210 validated instances spanning 9 Network Functions with a dual test strategy, Docker-based reproducibility, and an automated construction pipeline.

Our evaluation across four LLMs reveals that agentic iteration is the primary bottleneck, with multi-turn feedback improving resolve rates from 0% to 10 to 30%. The specification-as-skill experiment demonstrates that 3GPP protocol excerpts provide targeted improvement on specification-dependent bugs while offering no benefit for generic defensive checks, extending the findings of SWE-Skills-Bench to the telecommunications domain. The dataset, evaluation harness, and all experimental artifacts are publicly available at <https://huggingface.co/datasets/tenderzada/SWEBench5G>.

REFERENCES

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [2] Z. Shi *et al.*, “BeyondSWE: A comprehensive benchmark for evaluating code agents beyond narrow bug fixing,” *arXiv preprint arXiv:2603.03194*, 2025.
- [3] T. Han, Y. Zhang, W. Song, C. Fang, Z. Chen, Y. Sun, and L. Hu, “SWE-skills-bench: Do agent skills actually help in real-world software engineering?” *arXiv preprint arXiv:2603.15401*, 2025.
- [4] free5GC Team, “free5GC: Open source 5G core network,” <https://github.com/free5gc/free5gc>, 2024.
- [5] Z. Yao *et al.*, “SWE-bench mobile: An evaluation benchmark for mobile app engineering,” *arXiv preprint arXiv:2602.09540*, 2025.